

Linux y HPC para big data II

Alex Di Genova

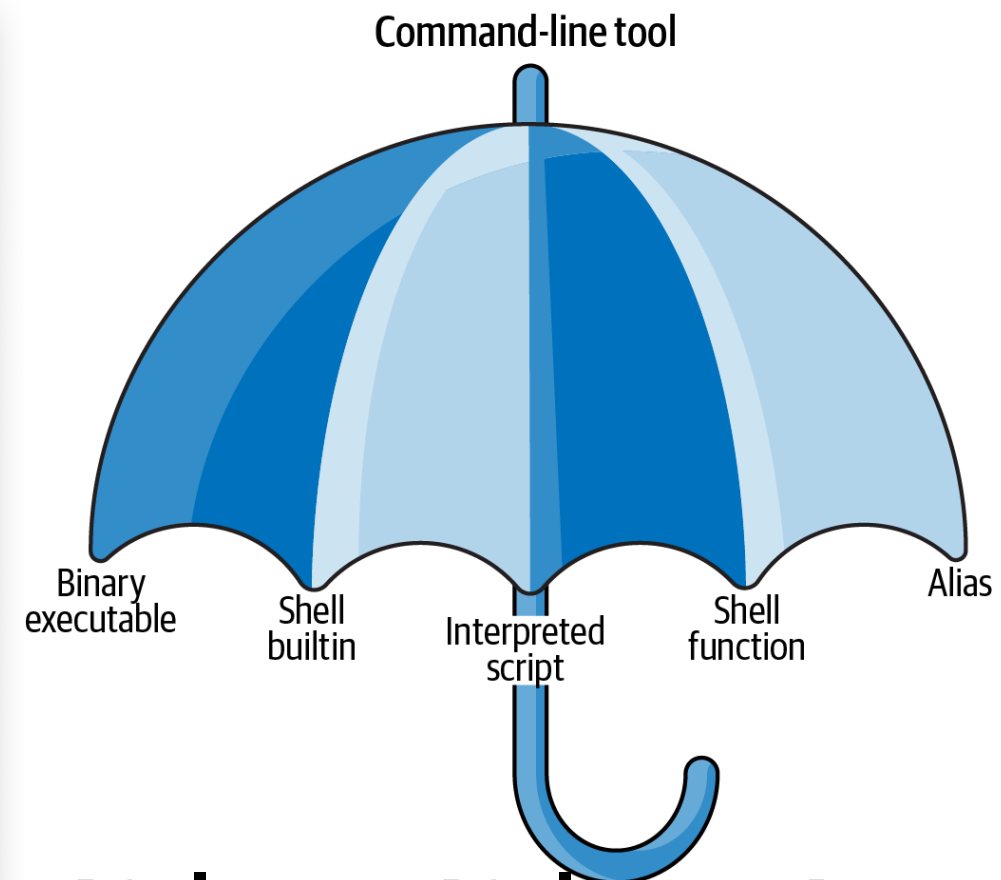
01/09/2026

Linux

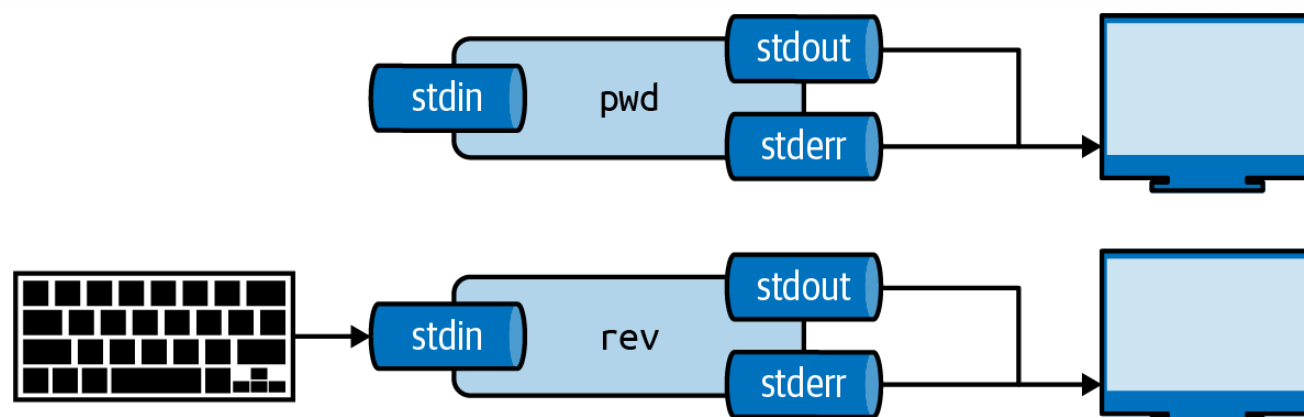
Shell command line

```
clases — adigenova@host04:~ — ssh adigenova@172.16.105.104 — 80x23
...@host04:~ — -bash ...
...R/pipelines — -bash ...
...ethylation — -bash ...
...trekka_pool — -bash ...
...ova@172.16.105.104 ...
...elka_pool] — -bash ...
...egfs/home — -bash ... +
[adigenova@host04 ~]$ echo "The command line is the force DCBI" | cowsay -f tux
< The command line is the force DCBI >
\
  .--.
  |o_o|
  |: _/|
  //   \ \
  (|   |)
  /'\_ _/\' \
  \_____)=(_____/

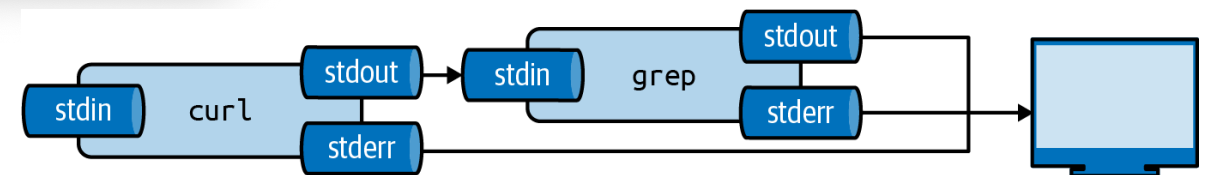
[adigenova@host04 ~]$
```



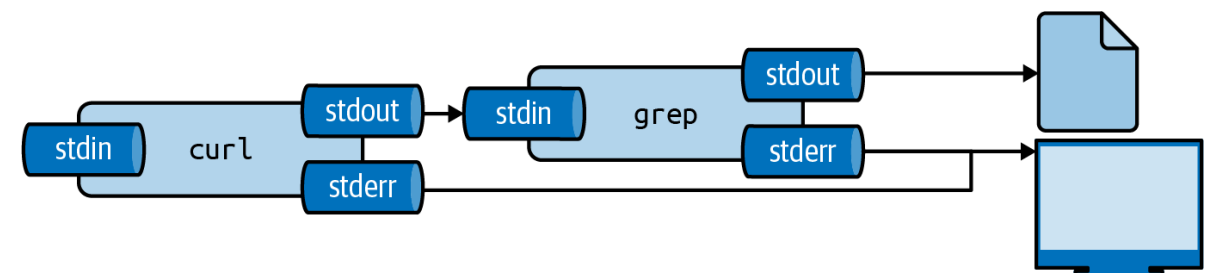
cmd1 | cmd2 | cmd3 | cmd4 ...



Every tool has three standard streams: standard input (`stdin`), standard output (`stdout`), and standard error (`stderr`)



The output from a tool can be piped to another tool



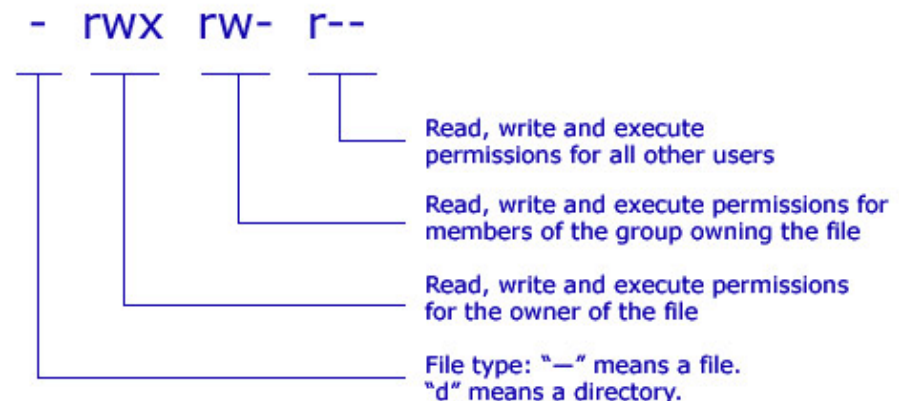
The output from a tool can be redirected to a file

Manipulating files

Linux

Everything is a file

- File types
 - Regular files (text, images)
 - Directories (list of files)
 - Symbolic links
 - Devices and peripherals
 - Pipes
 - Sockets (inter-process communication)
 - File names
 - Case sensitive, any character (/), extension not needed (but user?)
- File paths
 - Relative path: ../work/file1.txt
 - Absolute path: /home/adigenova/work/file1.txt
 - File access rights
 - Types
 - Read(r), write(w), execute(x)
 - Levels
 - User (u), Group (g), Other(o)



Linux

Everything is a file

- Common text files
 - CSV -> comma-separated values
 - TSV -> Tab-separated values
 - TXT -> text file (space separated)
 - xls|xlsx -> Excel files (text or binary?)
 - SQL database
- Getting data
 - `curl -LO https://burntsushi.net/stuff/worldcitiespop.csv`
 - `wget https://burntsushi.net/stuff/worldcitiespop.csv`
 - Tar `zxvf file.tar.gz`, `gzip` , `gz`, `zip` ... (compressed data)
 - `Scp`, `cp`, `rsync` ...
- Text file editors
 - VIM/Vi
 - Nano

Linux

Working with text files line by line

- Viewing and Creating Files: ***cat, tac, more, less, echo, touch***
- Editing Files: ***nano, vi, vim***
- File Content Manipulation:
 - ***grep***: Searches for patterns within files; can be used to find specific text.
 - ***sed***: Stream editor for filtering and transforming text in a file or input from a pipeline.
 - ***awk***: A programming language designed for text processing and manipulation.
 - ***paste***: Merges lines of files horizontally.
 - ***cut, sort, uniq, tr, wc, head, tail***.

Linux

Working with text files line by line

- Comparing Files:
 - **diff** : Compares files line by line.
- Regular Expressions and Pattern Matching: **grep, egrep, fgrep, sed, awk**
- Text Processing:
 - **join**: Joins lines of two files on a common field.
 - **split**: Splits a file into fixed-size pieces.
 - **nl** (add line numbers), **expand** (tab to spaces), **unexpand** (spaces to tab), **fold y fmt** (text format).
- File Encoding and Character Conversion: **dos2unix, unix2dos**

Linux

Working with text files line by line

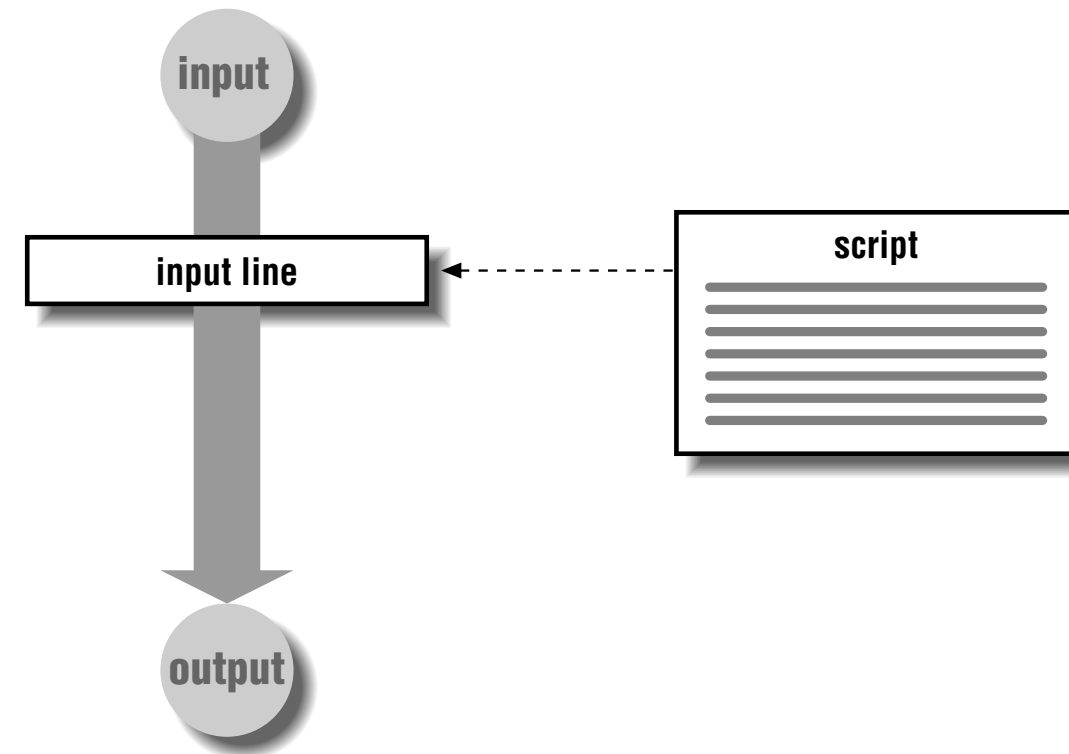
- `head worldcitiespop.csv`
- `wc -l worldcitiespop.csv`
- `head -100 worldcitiespop.csv > sample_worldcitiespop.csv`
- `cut -d',' -f1,5 worldcitiespop.csv > country_population.csv`
- `tail -n +2 worldcitiespop.csv | sort -t',' -k5 -nr | head -10`
- `cut -d',' -f1 worldcitiespop.csv | sort | uniq | tail -n +2`
- `awk -F',' 'NR>1 && $5 > 1000000' worldcitiespop.csv`
- `awk -F',' 'NF > 1 {pop[$1]+=$5} END{ for(c in pop){print c" "pop[c]}}'`
`worldcitiespop.csv | sort -rn -k2,2 | head`
- Add country name using join..

```
Country,City,AccentCity,Region,Population,Latitude,Longitude
ad,aixas,Aixàs,06,,42.4833333,1.4666667
ad,aixirivali,Aixirivali,06,,42.4666667,1.5
ad,aixirivall,Aixirivall,06,,42.4666667,1.5
ad,aixirvall,Aixirvall,06,,42.4666667,1.5
ad,aixovall,Aixovall,06,,42.4666667,1.4833333
ad,andorra,Andorra,07,,42.5,1.5166667
ad,andorra la vella,Andorra la Vella,07,20430,42.5,1.5166667
ad,andorra-vieille,Andorra-Vieille,07,,42.5,1.5166667
ad,andorre,Andorre,07,,42.5,1.5166667
```


Linux

Sed and Awk -- Power tools for editing files

- The basic function of `awk` is to search files for lines (or other units of text) that contain certain patterns.
 - A Pattern-Matching Programming Language
 - `awk 'program' input-file1 input-file2 ...`
 - View a text file as a textual database made up of records and fields.
 - Use variables to manipulate the database.
 - Use arithmetic and string operators.
 - Use common programming constructs such as loops and conditionals.
 - Generate formatted reports.
- Sed
 - Sed is a “non-interactive” stream-oriented editor.
 - To automate editing actions to be performed on one or more files.
 - To simplify the task of performing the same edits on multiple files.
 - To write conversion programs.



```
sed 's/2022/2024/g' file.txt  
sed 's/2022/2024/g ; s/casa/home/g' file.txt
```

```
awk '{print $1}' file.txt  
awk '{print $1}' *.txt  
awk '/perro/ {print $0}' *.txt
```

Linux

Working with text files line by line

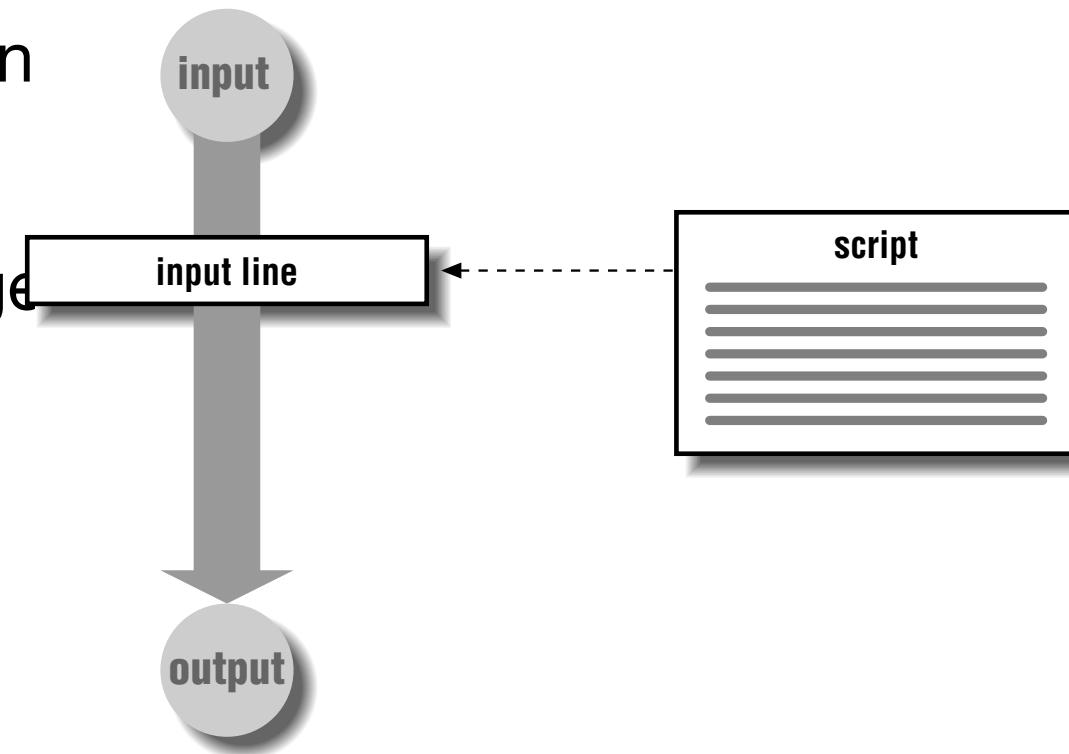
- `head worldcitiespop.csv`
- `wc -l worldcitiespop.csv`
- `head -100 worldcitiespop.csv > sample_worldcitiespop.csv`
- `cut -d',' -f1,5 worldcitiespop.csv > country_population.csv`
- `tail -n +2 worldcitiespop.csv | sort -t',' -k5 -nr | head -10`
- `cut -d',' -f1 worldcitiespop.csv | sort | uniq | tail -n +2`
- `awk -F',' 'NR>1 && $5 > 1000000' worldcitiespop.csv`
- `awk -F',' 'NF > 1 {pop[$1]+=$5} END{ for(c in pop){print c" "pop[c]}}'`
`worldcitiespop.csv | sort -rn -k2,2 | head`
- Add country name using join..

```
Country,City,AccentCity,Region,Population,Latitude,Longitude
ad,aixas,Aixàs,06,,42.4833333,1.4666667
ad,aixirivali,Aixirivali,06,,42.4666667,1.5
ad,aixirivall,Aixirivall,06,,42.4666667,1.5
ad,aixirvall,Aixirvall,06,,42.4666667,1.5
ad,aixovall,Aixovall,06,,42.4666667,1.4833333
ad,andorra,Andorra,07,,42.5,1.5166667
ad,andorra la vella,Andorra la Vella,07,20430,42.5,1.5166667
ad,andorra-vieille,Andorra-Vieille,07,,42.5,1.5166667
ad,andorre,Andorre,07,,42.5,1.5166667
```

Linux

Sed and Awk — - Power tools for editing files

- The basic function of `awk` is to search files for lines (or other units of text) that contain certain patterns.
- A Pattern-Matching Programming Language
- `awk 'program' input-file1 input-file2 ...`
- View a text file as a textual database made up of records and fields.
- Use variables to manipulate the database.
- Use arithmetic and string operators.
- Use common programming constructs such as loops and conditionals.
- Generate formatted reports.



Linux

Sed and Awk —- Power tools for editing files

- Sed

- Sed is a “non-interactive” stream-oriented editor.
- To automate editing actions to be performed on one or more files.
- To simplify the task of performing the same edits on multiple files.
- To write conversion programs.

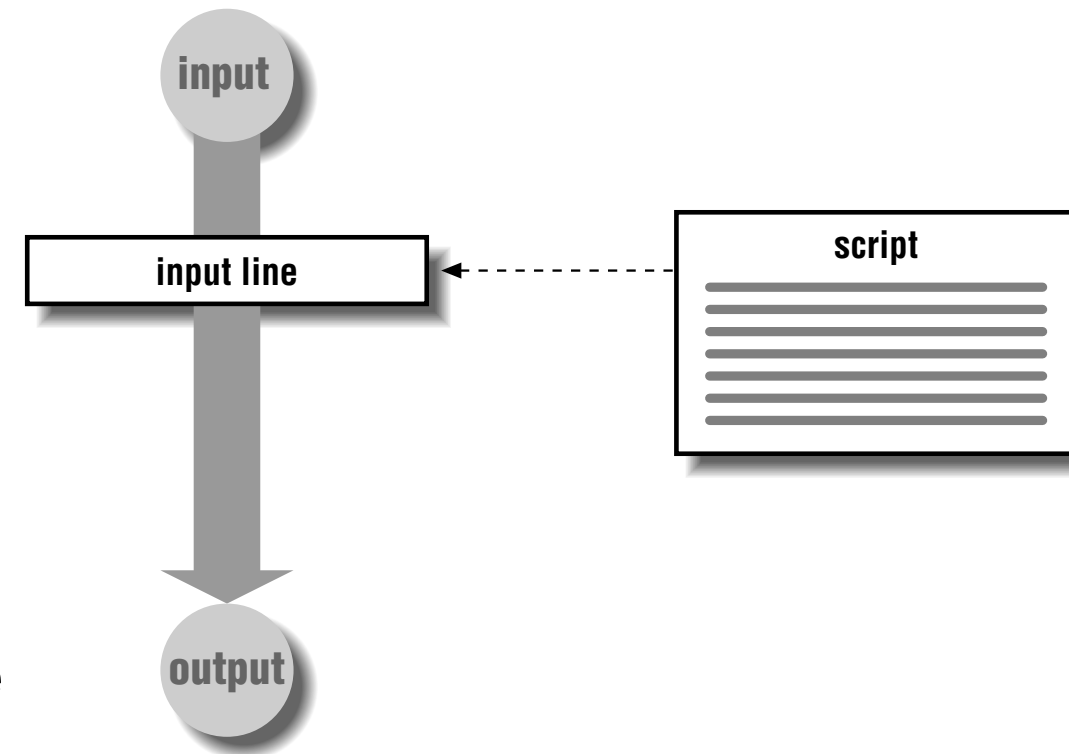
```
sed 's/2022/2024/g' file.txt
```

```
sed 's/2022/2024/g ; s/casa/home/g' file.txt
```

```
awk '{print $1}' file.txt
```

```
awk '{print $1}' *.txt
```

```
awk '/perro/ {print $0}' *.txt
```



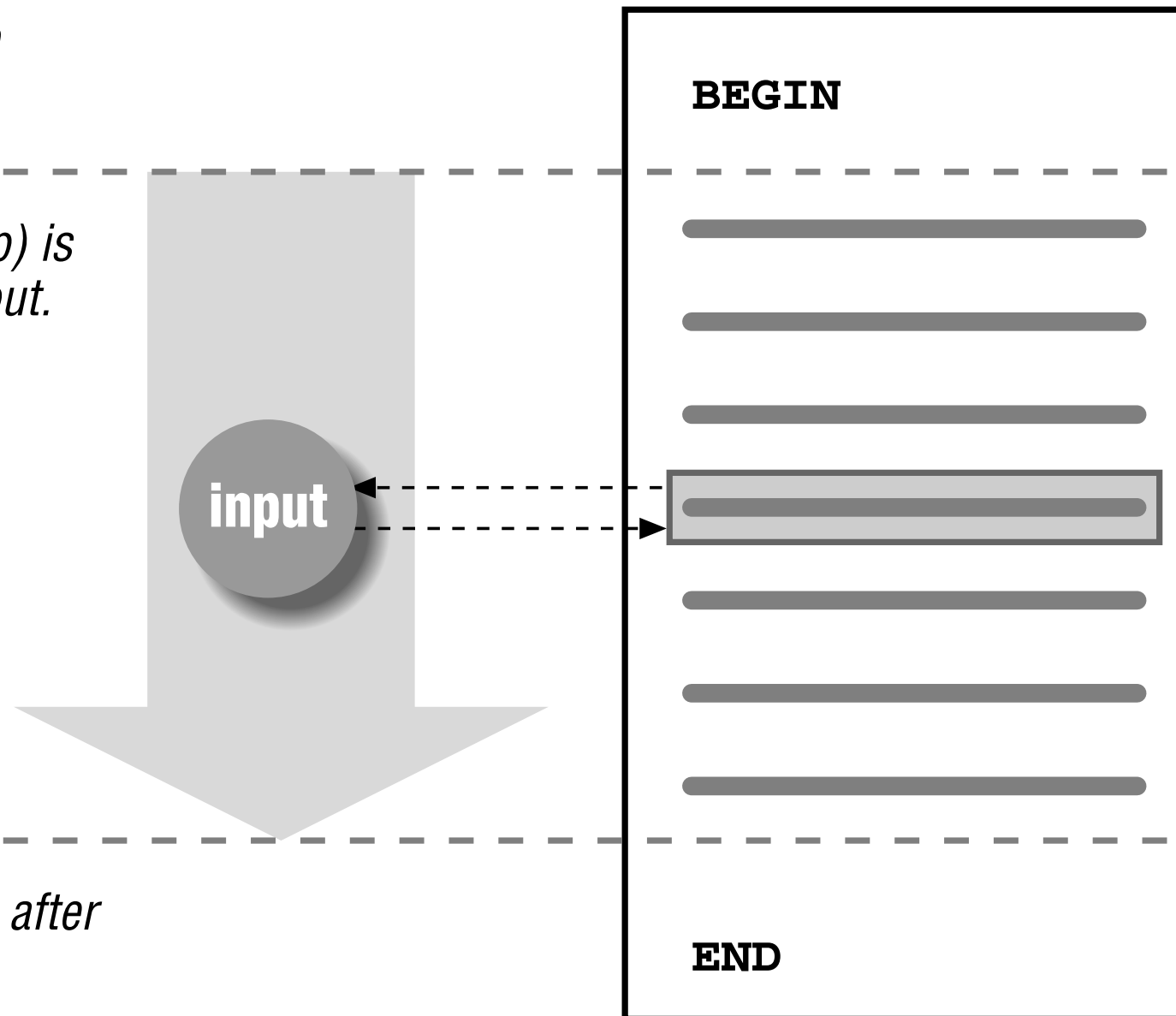
Linux

Awk

1 *1st routine is executed once before any input is read.*

2 *2nd routine (main input loop) is executed for each line of input.*

3 *3rd routine is executed once after all input is read.*



Linux

Awk

- *if (expression) action1; [else action2]*
expr ? action1 : action2
grade = (avg >= 65) ? "Pass" : "Fail"
- array[subscript] = value*
for (variable in array)
do something with array[variable]

while (condition)

action

for (set_counter ; test_counter ; increment_counter)

action

Awk Function	Description
<code>cos(<i>x</i>)</code>	Returns cosine of <i>x</i> (<i>x</i> is in radians).
<code>exp(<i>x</i>)</code>	Returns <i>e</i> to the power <i>x</i> .
<code>int(<i>x</i>)</code>	Returns truncated value of <i>x</i> .
<code>log(<i>x</i>)</code>	Returns natural logarithm (base- <i>e</i>) of <i>x</i> .
<code>sin(<i>x</i>)</code>	Returns sine of <i>x</i> (<i>x</i> is in radians).
<code>sqrt(<i>x</i>)</code>	Returns square root of <i>x</i> .
<code>atan2(<i>y</i>,<i>x</i>)</code>	Returns arctangent of <i>y</i> / <i>x</i> in the range $-\pi$ to π .
<code>rand()</code>	Returns pseudo-random number <i>r</i> , where $0 \leq r < 1$.
<code>srand(<i>x</i>)</code>	Establishes new seed for <code>rand()</code> . If no seed is specified, uses time of day. Returns the old seed.

Linux

Awk

Awk Function	Description
<code>gsub(<i>r,s,t</i>)</code>	Globally substitutes <i>s</i> for each match of the regular expression <i>r</i> in the string <i>t</i> . Returns the number of substitutions. If <i>t</i> is not supplied, defaults to <code>\$0</code> .
<code>index(<i>s,t</i>)</code>	Returns position of substring <i>t</i> in string <i>s</i> or zero if not present.
<code>length(<i>s</i>)</code>	Returns length of string <i>s</i> or length of <code>\$0</code> if no string is supplied.
<code>match(<i>s,r</i>)</code>	Returns either the position in <i>s</i> where the regular expression <i>r</i> begins, or 0 if no occurrences are found. Sets the values of RSTART and RLENGTH .
<code>split(<i>s,a,sep</i>)</code>	Parses string <i>s</i> into elements of array <i>a</i> using field separator <i>sep</i> ; returns number of elements. If <i>sep</i> is not supplied, FS is used. Array splitting works the same way as field splitting.
<code>sprintf("<i>fmt</i>",<i>expr</i>)</code>	Uses printf format specification for expr .
<code>sub(<i>r,s,t</i>)</code>	Substitutes <i>s</i> for first match of the regular expression <i>r</i> in the string <i>t</i> . Returns 1 if successful; 0 otherwise. If <i>t</i> is not supplied, defaults to <code>\$0</code> .
<code>substr(<i>s,p,n</i>)</code>	Returns substring of string <i>s</i> at beginning position <i>p</i> up to a maximum length of <i>n</i> . If <i>n</i> is not supplied, the rest of the string from <i>p</i> is used.
<code>tolower(<i>s</i>)</code>	Translates all uppercase characters in string <i>s</i> to lowercase and returns the new string.
<code>toupper(<i>s</i>)</code>	Translates all lowercase characters in string <i>s</i> to uppercase and returns the new string.

Awk

Examples

```
$ cat table.txt  
brown bread mat hair 42  
blue cake mug shirt -7  
yellow banana window shoes 3.14
```

```
# print the second field of each input line
```

```
$ awk '{print $2}' table.txt  
bread  
cake  
banana
```

```
# print lines only if the last field is a negative number
```

```
# recall that the default action is to print the contents of $0
```

```
$ awk '$NF<0' table.txt  
blue cake mug shirt -7
```

```
# change 'b' to 'B' only for the first field
```

```
$ awk '{gsub(/b/, "B", $1)} 1' table.txt  
Brown bread mat hair 42  
Blue cake mug shirt -7  
yellow banana window shoes 3.14
```

```
awk 'cond1{action1} cond2{action2} ... condN{actionN}'
```

https://learnbyexample.github.io/learn_gnuawk/

Linux

Sed

- Substitutions
 - /Pattern/replacement/flags
 - Flags : n <replace the n-matchin pattern>,g<global>,l <insensitive case>,p <print pattern>
- Transform
 - s/abc/xyz/
 - y/abcdefghijklmnopqrstuvwxyz/ABCDEFGHIJKLMNOPQRSTUVWXYZ/
- Regular expressions with Sed
 - sed 's/^(.*\):\(.*\)-\(.*\)^3:\2:\1/' test.txt

1:101-201	201:101:1
2:102-202	202:102:2
3:103-203	203:103:3
4:104-204	204:104:4
5:105-205	205:105:5
6:106-206	206:106:6

Linux

Regular expressions

Basic Syntax

- `/.../`: Start and end
- `|`: Alternation
- `()`: Grouping

Groups and Ranges

- `.`: Any character except newline (`\n`)
- `(alb)`: a or b
- `(...)`: Group
- `(?:...)`: Passive (non-capturing) group
- `[abc]`: a, b or c
- `[^abc]`: Not a, b or c
- `[a-z]`: Letters from a to z
- `[A-Z]`: Uppercase letters from A to Z
- `[0-9]`: Digits from 0 to 9

Position Matching

- `^`: Start of string or start of line
- `$`: End of string or end of line
- `\b`: Word boundary
- `\B`: Not word boundary

Character Classes

- `\s`: Whitespace
- `\S`: Not whitespace
- `\w`: Word
- `\W`: Not word
- `\d`: Digit
- `\D`: Not digit
- `\x`: Hexadecimal digit
- `\O`: Octal digit

Quantifiers

- `*`: 0 or more
- `+`: 1 or more
- `?`: 0 or 1
- `{3}`: Exactly 3
- `{3,}`: 3 or more
- `{3,5}`: 3, 4 or 5



<https://regexr.com/>

Linux

Sed & regex examples

```
$ cat anchors.txt
sub par
spar
apparent effort
two spare computers
cart part tart mart

# words starting with 'par'
$ sed -n '/\bpar/p' anchors.txt
sub par
cart part tart mart

# words ending with 'par'
$ sed -n '/par\b/p' anchors.txt
sub par
spar

$ sed -nE '/two|sub/p' anchors.txt
sub par
two spare computers

# match 'cat' or 'dog' or 'fox'
# note the use of 'g' flag for mul
$ echo 'cats dog bee parrot foxed' | sed -E 's/cat|dog|tox/--/g'
--s -- bee parrot --ed

$ echo 'ac abc abbc abbbc abbbbbbbbc' | sed -E 's/ab{1,4}c/X/g'
ac X X X abbbbbbbbc

$ echo 'ac abc abbc abbbc abbbbbbbbc' | sed -E 's/ab{3,}c/X/g'
ac abc abbc X X

$ echo 'ac abc abbc abbbc abbbbbbbbc' | sed -E 's/ab{,2}c/X/g'
X X X abbbc abbbbbbbbc

$ echo 'ac abc abbc abbbc abbbbbbbbc' | sed -E 's/ab{3}c/X/g'
ac abc abbc X abbbbbbbbc

$ echo 'fd fed fod fe:d feeeder' | sed 's/fe*d/X/g'
X X fod fe:d Xer

# zero or more of '1' followed by '2'
$ echo '311111111125111142' | sed 's/1*2/-/g'
3-511114-

# from the first 'b' to the last 't' in the line
$ echo 'car bat cod map scat dot abacus' | sed 's/b.*t/-/'
car - abacus

# from the first 'b' to the last 'at' in the line
$ echo 'car bat cod map scat dot abacus' | sed 's/b.*at/-/'
car - dot abacus
```